

课程项目：菜鸟驿站配送调度系统

一、项目背景

菜鸟驿站的送货小车总在校园里忙忙碌碌，但它看起来不太聪明。本项目要你把它变聪明一点：读入“校园地图、待配送的包裹和小车参数”，**规划送货路线、完成配送，并算出各项指标。**

数据从文件读入，格式见《数据格式说明.md》。核心概念：

- **地图**：一个无向带权图，节点是地点，边权是距离；**0 号节点是驿站**，小车从这里出发、最终回到这里。
- **包裹**：每个包裹有重量 `w`、目的地、到站（可派送）时间 `s`、最晚送达时间 `T`。
- **小车**：有速度 `v`、自重 `w_car`、装载重量上限 `w_lim`；一次装载不能超过 `w_lim`，装不下就得多跑几趟。

二、需要实现的数据结构

1. **图**（邻接表或邻接矩阵，自选，并在报告中说明为何这样选）；
2. **最小堆 / 优先队列**（供 Dijkstra 使用——**不要用** `std::priority_queue`，代码中请手写实现）；
3. **队列**（用于维护驿站的待配送任务，例如循环队列）。

报告中需给出每个结构与算法的时间复杂度分析。

三、任务

在开始各任务前，先明确**配送的基本规则**（除非某个任务另有说明，否则均遵守以下规则）：

- 全程只有一辆小车（仅 T5 会增加第二辆）；小车从驿站（0 号节点）出发装货。
- 小车沿地图的边行驶：走过一条长度为 L 的边耗时 L/v （ v 为速度）；装卸货物本身不计时间。
- 小车一趟所装包裹的重量之和不能超过上限 `w_lim`；装不下的留到下一趟运输。
- 一趟 = 从驿站装货出发 → 依次开到各目的地卸货 → 返回驿站；需要多趟就重复这个过程。
- 包裹 i 的**到站时间** S_i ，指包裹到达驿站的时间，在该时刻开始，该包裹即可配送。
- 包裹 i 的**送达时间** D_i ，指小车到达它的目的地、把它卸下的那一刻。
- 送完全部包裹后，小车最终回到驿站。

必做

T1 最短路查询

配送的基础是“两点之间怎么走最近”。请实现 **Dijkstra 单源最短路**（用你自己写的最小堆），能查询地图上任意两个节点间的**最短距离及其具体路径**（依次经过哪些节点）。

要求：至少能查询“驿站(0) → 任意节点”的最短距离与路径——这也是 T2、T3 里计算“从一个点到另一个点要走多远、花多久”的基础。示例数据的标准答案见 `测试数据/示例/预期结果.md`，可用来自检你的实现。

T2 最小化“不满意度之和”

新增条件：

- 本任务**不考虑超时**：可认为每个包裹的最晚送达时间 T_i 都是无穷大（数据中以一个很大的数如 `1e9` 给出）。
- 每个包裹只有在它到站时间 S_i 之后才可被派送：小车某一趟从驿站出发（去送本趟包裹）的时刻，必须不早于本趟所载每个包裹的 S_i ——还没到站的包裹这一趟带不了，只能留到下一趟运输。

- 仍受容量上限 `w_lim` 约束，可能需要多趟运输。

定义：包裹 i 的不满意度 = $D_i - S_i$ ，即它“从到站到被送达一共等了多久”（ D_i 为送达时间）。

要求：规划一种送货方案（每趟带哪些包裹、以什么顺序送、共几趟），使**所有包裹的不满意度之和** $\sum(D_i - S_i)$ 尽量小。输出你的方案 and 这个总和。

T3 带容量的运送成本

新增条件：

- 每个包裹的到站时间都视为 0（即 $S_i = 0$ ），任何包裹随时都可派送。
- 容量上限 `w_lim` 生效，可能需要多趟。
- 每个包裹有最晚送达时间 T_i ；若其送达时间 $D_i > T_i$ ，记为一次**超时**。

定义（运送总成本）：小车每走过一段路，这段的代价 = 段长 $\times$ （小车自重 `w_car` + 这段路上车里还没卸下的包裹总重）；把所有趟、所有路段的代价累加即为总成本：

成本 = $\sum(\text{每一趟}) \sum(\text{该趟经过的每一段路}) \text{ 段长} \times (\text{w_car} + \text{该段行驶时车上的包裹总重})$

也就是说：车越重越费力，装得越多、跑得越远越贵；每到一个目的地卸下包裹后，后续路段的车载重量随之减小；空车返回驿站的路段包裹重为 0，但仍要计入自重 `w_car`。一个逐段计算的具体算例见 `测试数据/示例/预期结果.md`。

要求：规划一种送货方案，输出你的方案、**运送总成本**，以及**超时包裹的数量**（即 $D_i > T_i$ 的包裹个数）。

提示：T1 / T2 / T3 相互独立，都是在最原始数据基础上按各自“新增条件”扩展、互不影响（例如 T3 不必考虑 T2 的不满意度）。

选做（加分，总分 100）

以下每完成一项都可加分，请在报告中**量化对比**改进效果：

T4 退货回收

新增条件：本任务小车最大载重视为无穷大（`1e9`），容量不再是约束；另给定若干**需要退货的地点**（节点）。

要求：小车从驿站出发，把这些地点的退货**全部取回并送回驿站**，规划一条路线使**总耗时最短**。这本质是旅行商问题(TSP)——要决定以什么顺序访问这些点：地点较少时可用回溯 / 状态压缩 DP 求精确最优，较多时用启发式。（退货地点用什么方式给出可自行约定，或直接复用现有数据里的若干目的地，请在报告中说明。）

T5 双车协同

新增条件：有**两辆**参数相同的小车，可同时从驿站出发、各自装货与配送。

要求：把包裹分配给两辆车并各自规划路线，成本定义与 T3 相同，使**总成本尽量小**，并统计超时数。越优越好。

T6 更优策略 + 自造数据

前面的 T2 / T3 只要求你给出一种可行且尽量好的配送策略。本任务要求你进一步比较不同策略的效果，说明“为什么某些策略在某些数据上更好”。

要求：

1. 至少实现 **1 种基线策略** 和 **1 种改进策略**。例如：
 - 基线策略：按包裹编号顺序装车、按最近目的地优先、按最早截止时间优先等；
 - 改进策略：综合考虑距离、重量、等待时间、截止时间的评分策略。
2. 至少自己构造 **2 组测试数据**，并说明每组数据想验证什么。例如：
 - 一组让“最近优先”表现较好；
 - 一组让“只看最近距离”表现较差，而综合策略更好。
3. 对每组数据，分别运行不同策略，并用表格比较结果。比较指标至少包含以下一种：
 - T2 的不满意度之和；

- T3 的运送总成本；
- T3 的超时包裹数量。

4. 在报告中简要分析结果：哪些策略更好、好在哪里、是否存在某些数据让它反而变差。

本任务不要求证明你的策略是全局最优，重点是能设计合理的策略、构造有区分度的数据，并用实验结果解释策略差异。

结果可视化

用你会的任意方式（控制台表格、简单图形界面等，**自行设计**）把地图与配送过程展示出来。

四、交付物

1. **全部由你编写的 C++ 源代码**（能用 `g++ -std=c++17` 编译，能读入《数据格式说明》规定的数据）；
2. 能体现**逐步开发过程**的内容，比如说 git 的提交历史等。
3. **项目报告**：
 - 你的数据结构设计与**复杂度分析**；
 - 各配送任务的思路、正确性说明；
 - 在给定测试数据上跑出的结果（截图或表格）；
 - 选做部分的量化对比（如有）；
4. **AI_USAGE（AI 使用日志，必交）**：你用 AI 做了什么、AI 哪里给错了你如何发现并修正、哪部分你最有把握 / 最没把握。**允许使用 AI，但要大致讲清楚实现任务的基本逻辑和使用的算法或者方案。**

五、测试数据

测试数据/ 目录下提供了几组数据：

- 示例/：一个 6 节点小地图，附 预期结果.md（最短路距离、成本公式算例）
- 测试1/ 测试2/ 测试3/：规模递增的数据，用来跑你的程序、写进报告。

评分时我们会用**同样格式、但不同内容**的数据（可能是你的专属数据）来运行你的程序，所以请确保你的程序严格按《数据格式说明》读入。